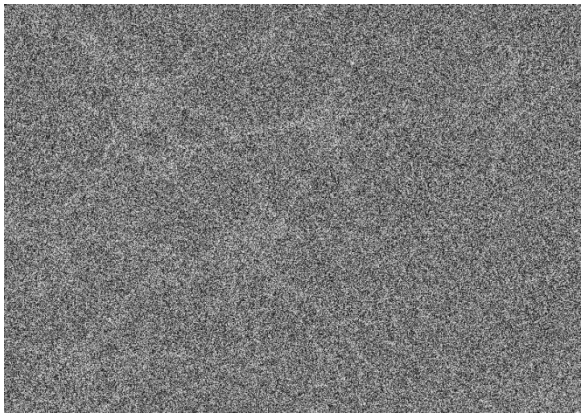
Mini Project # 4

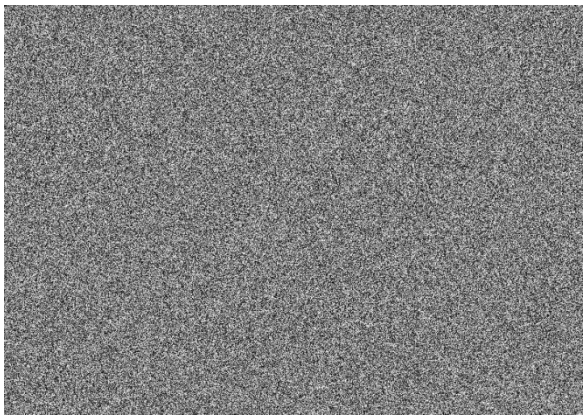
MOHAMMED IMTIYAZ

FAU ID: Z23749265

The following two images are captured from a biological specimen. Both images are mixed and distorted by the same random image as shown in the figures.



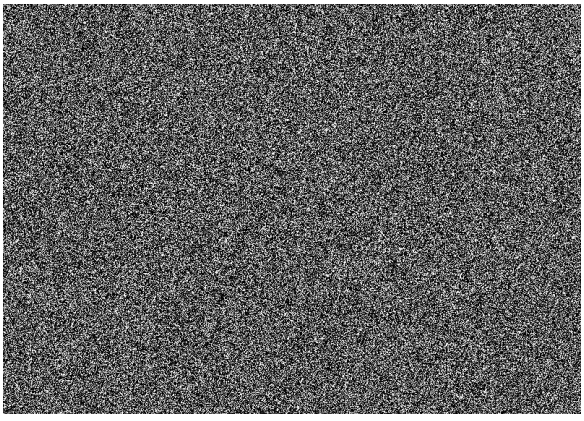
Recording # 1

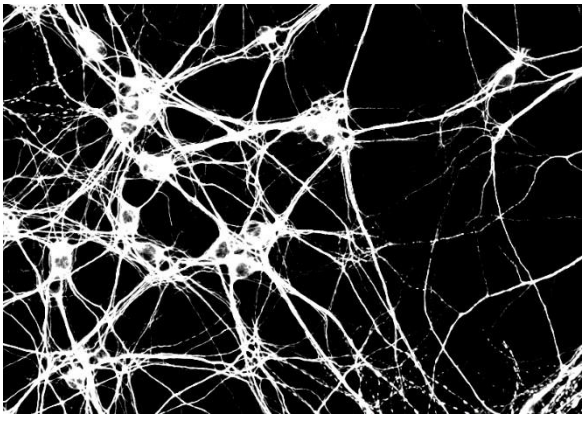


Recording # 2

Download these two images (Recording-1.png and Recording-2.png) from Canvas. Then use the ICA algorithm to separate the distortion from the main image. Implement Fast-ICA algorithm for noise separation.

Solution:





Sol:

Independent Component Analysis is one of the various unsupervised learning algorithms which means that we do not need to supervise the model before we can use it.

Code:

```
import numpy as np
from sklearn.decomposition import FastICA
from PIL import Image
import matplotlib.pyplot as plt

image1_path = '/content/drive/MyDrive/My/Machine learning Files/Recording-
1.png'
image2_path = '/content/drive/MyDrive/My/Machine learning Files/Recording-
2.png'

def load_image(path):
    # Function to load an image and convert to grayscale
    return np.array(Image.open(path).convert('L'))

image1_array = load_image(image1_path)
image2_array = load_image(image2_path)

X = np.stack([image1_array.flatten(), image2_array.flatten()], axis=-1)

# Normalize the data to have a mean of 0 and a standard deviation of 1.
# This is done to improve the performance of the ICA algorithm.
# We add a small constant (1e-5) to avoid division by zero errors.
X_normalized = (X - np.mean(X, axis=0)) / (np.std(X, axis=0) + 1e-5)

# Ensure 'arbitrary-variance' is specified for whiten
ica = FastICA(n_components=2, random_state=0, whiten='arbitrary-variance')
X_transformed = ica.fit_transform(X_normalized)

image1_separated = X_transformed[:, 0].reshape(image1_array.shape)
image2_separated = X_transformed[:, 1].reshape(image2_array.shape)

threshold1 = (image1_separated.min() + image1_separated.max()) / 2
threshold2 = (image2_separated.min() + image2_separated.max()) / 2

image1_thresholded = image1_separated <= threshold1
image2_thresholded = image2_separated <= threshold2

fig, axes = plt.subplots(1, 2, figsize=(15, 5))
```

```
axes[0].imshow(image1_thresholded, cmap='gray')
axes[0].axis('off')

axes[1].imshow(image2_thresholded, cmap='gray')
axes[1].axis('off')

plt.show()
```

output:

